The STATE[4] Pattern

Still another technique for implementing finite state machines is the STATE pattern. This pattern combines the efficiency of the nested switch/case statement with the flexibility of interpreting a transition table.

Figure 29-3 shows the structure of the solution. The `Turnstile` class has public methods for the events and protected methods for the actions. It holds a reference to an interface called `TurnstileState`. The two derivatives of `TurnstileState` represent the two states of the FSM.

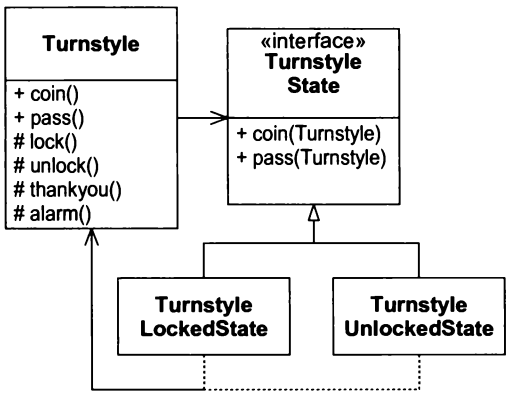


Figure 29-3 The STATE Pattern for the Turnstile

When one of the two event methods of `Turnstile` is invoked, it delegates that event to the `TurnstileState` object. The methods of `TurnstileLockedState` implement the appropriate actions for the LOCKED state. The methods of `TurnstileUnlockedState` implement the appropriate actions for the UNLOCKED state. To change the state of the FSM, the reference in the `Turnstile` object is assigned to an instance of one of these derivatives.

Listing 29-6 shows the `TurnstileState` interface and its two derivatives. The state machine is easily visible in the four methods of those derivatives. For example, the `coin` method of `LockedTurnstileState` tells the `Turnstile` object to change state to the unlocked state, and then it invokes the `unlock` action function of `Turnstile`.

Listing 29-6

`TurnstileState.java`

```
interface TurnstileState
{
  void coin(Turnstile t);
  void pass(Turnstile t);
}

class LockedTurnstileState implements TurnstileState
{
  public void coin(Turnstile t)
  {
    t.setUnlocked();
    t.unlock();
  }
```

4. [GOF95], p. 305.

```
  public void pass(Turnstile t)
  {
    t.alarm();
  }
}

class UnlockedTurnstileState implements TurnstileState
{
  public void coin(Turnstile t)
  {
    t.thankyou();
  }

  public void pass(Turnstile t)
  {
    t.setLocked();
    t.lock();
  }
}
```

The `Turnstile` class is shown in Listing 29-7. Notice the static variables that hold the derivatives of `TurnstileState`. These classes have no variables and therefore never need to have more than one instance. Holding the derivative instances of the `TurnstileState` derivatives in variables obviates the need to create a new instance every time the state changes. Making those variables static obviates the need to create new instances of the derivatives in the event that we need more than one instance of `Turnstile`.

Listing 29-7

`Turnstile.java`

```
public class Turnstile
{
  private static TurnstileState lockedState = new LockedTurnstileState();
  private static TurnstileState unlockedState = new UnlockedTurnstileState();

  private TurnstileController turnstileController;
  private TurnstileState state = lockedState;

  public Turnstile(TurnstileController action)
  {
    turnstileController = action;
  }

  public void coin()
  {
    state.coin(this);
  }

  public void pass()
  {
    state.pass(this);
  }
```

```
  public void setLocked()
  {
    state = lockedState;
  }

  public void setUnlocked()
  {
    state = unlockedState;
  }

  public boolean isLocked()
  {
    return state == lockedState;
  }

  public boolean isUnlocked()
  {
    return state == unlockedState;
  }

  void thankyou()
  {
    turnstileController.thankyou();
  }

  void alarm()
  {
    turnstileController.alarm();
  }

  void lock()
  {
    turnstileController.lock();
  }

  void unlock()
  {
    turnstileController.unlock();
  }
}
```

STATE vs. STRATEGY. The diagram in Figure 29-3 is strongly reminiscent of the STRATEGY[5] pattern. Both have a context class; both delegate to a polymorphic base class that has several derivatives. The difference (see Figure 29-4) is that, in STATE, the derivatives hold a reference back to the context class. The primary function of the derivatives is to select and invoke methods of the context class through that reference. In the STRATEGY pattern, no such constraint or intent exists. The derivatives of a STRATEGY are not required to hold a reference to the context, and they are not required to call methods on the context. Thus, all instances of the STATE pattern are also instances of the STRATEGY pattern, but not all instances of STRATEGY are STATE.

Costs and Benefits of the STATE Pattern. The STATE pattern provides a very strong separation between the actions and the logic of the state machine. The actions are implemented in the `Context` class, and the logic is distributed through the derivatives of the `State` class. This makes it very simple to change one without affecting the other. For example, it would be very easy to reuse the actions of the `Context` class with a different state logic

5. See "Strategy" on page 168.

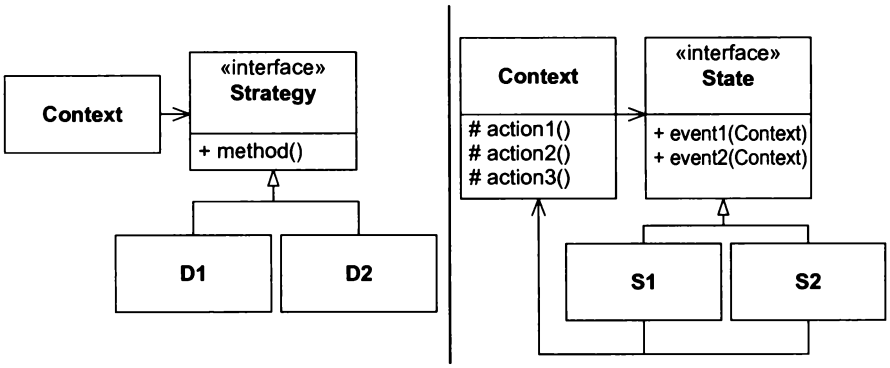


FIGURE 29-4 STATE V. STRATEGY

by simply using different derivatives of the `State` class. Alternatively we could create subclasses of `Context` that modify or replace the actions without affecting the logic of the `State` derivatives.

Another benefit of the this technique is that it is very efficient. It is probably just as efficient as the nested switch/case implementation. Thus, we have the flexibility of the table-driven approach with the efficiency of the nested switch/case approach.

The cost of this technique is twofold. First, the writing of the `State` derivatives is tedious at best. Writing a state machine with 20 states can be mind numbing. Second, the logic is distributed. There is no single place to go to see it all. This makes the code hard to maintain. This is reminiscent of the obscurity of the nested switch/case approach.

SMC—The State-Machine Compiler

The tedium of writing the derivatives of state and the need to have a single place to express the logic of the state machine led me to write a compiler that translates a textual state transition table into the classes necessary to implement the STATE pattern. This compiler is free and can be downloaded from `http://www.objectmentor.com`.

The input to the compiler is shown in Listing 29-8. The syntax is as follows:

```
currentState
{
  event newState action
  ...
}
```

The four lines at the top describe the name of the state machine, the name of the context class, the initial state, and the name of the exception that will be thrown in the event of an illegal event.

Listing 29-8

`Turnstile.sm`

```
FSMName Turnstile
Context TurnstileActions
Initial Locked
Exception FSMError
{
    Locked
    {
        coin    Unlocked    unlock
        pass    Locked      alarm
    }
```

```
    Unlocked
    {
        coin    Unlocked    thankyou
        pass    Locked      lock
    }
}
```

In order to use this compiler, you must write a class that declares the action functions. The name of this class is specified in the `Context` line. I called it `TurnstileActions`. (See Listing 29-9.)

Listing 29-9

`TurntstyleActions.java`

```
public abstract class TurnstileActions
{
  public void lock() {}
  public void unlock() {}
  public void thankyou() {}
  public void alarm() {}
}
```

The compiler generates a class that derives from the context. The name of the generated class is specified in the `FSMName` line. I called it `Turnstile`.

I could have implemented the action functions in `TurnstileActions`. However, I am more inclined to write another class that derives from the generated class and implements the action functions there. This is shown in Listing 29-10.

Listing 29-10

`TurnstileFSM.java`

```
public class TurnstileFSM  extends Turnstile
{
  private TurnstileController controller;
  public TurnstileFSM(TurnstileController controller)
  {
    this.controller = controller;
  }

  public void lock()
  {
    controller.lock();
  }

  public void unlock()
  {
    controller.unlock();
  }

  public void thankyou()
  {
    controller.thankyou();
  }
```

```
  public void alarm()
  {
    controller.alarm();
  }
}
```

That's all we have to write. SMC generates the rest. The resulting structure is shown in Figure 29-5. We call this a THREE-LEVEL FINITE STATE MACHINE.[6]

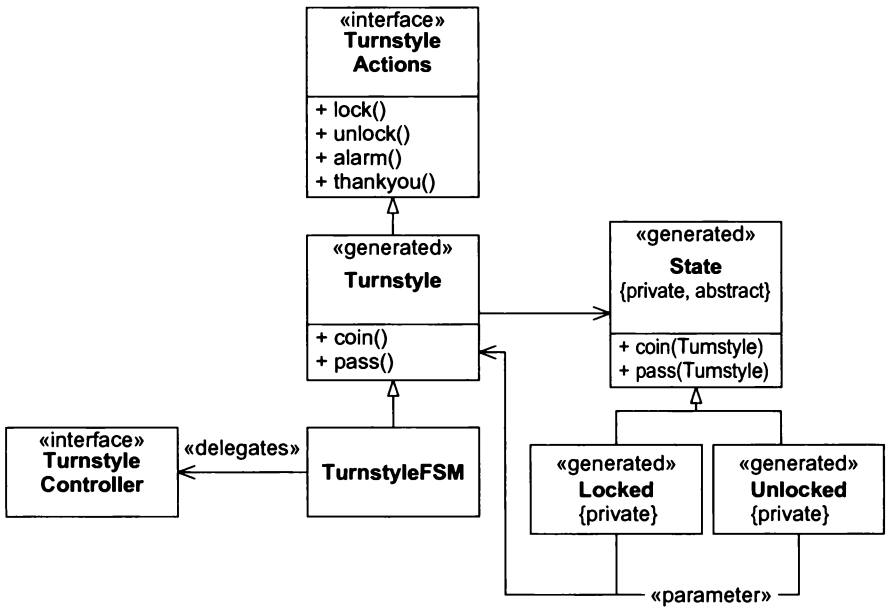


Figure 29-5 Three-Level FSM

The three levels provide the maximum in flexibility at a very low cost. We can create many different finite state machines simply by deriving them from `TurnstileActions`. We can also implement the actions in many different ways simply by deriving from `Turnstile`.

Notice that the generated code is completely isolated from the code that you have to write. You never have to modify the generated code. You don't even have to look at it. You can pay it the same level of attention that you pay to binary code.

You can see the generated code, as well as the other support code for this example, in Listing 29-13 through Listing 29-15 in the Listings section at the end of the chapter.

Costs and Benefits of the SMC Approach to the STATE Pattern. Clearly we have managed to maximize the benefits of the various approaches. The description of the finite state machine is contained all in one place and is very easy to maintain. The logic of the finite state machine is strongly isolated from the implementation of the actions, enabling both to be changed without impact upon the other. The solution is efficient, elegant, and requires a minimum of coding.

The cost is in the use of SMC. You have to have, and to learn how to use, another tool. In this case, however, the tool is remarkably simple to install and use. (See Listing 29-16 and the preceding paragraphs.) And it's free!

6. [PLoPD1], p. 383.

Where Should State Machines Be Used?

I use state machines (and SMC) for several different classes of applications.

High-Level Application Policies for GUIs

One of the goals of the graphical revolution in the 1980s was to create *stateless* interfaces for humans to use. At the time, computer interfaces were dominated by textual approaches using hierarchical menus. It was easy to get lost in the menu structure, not knowing what *state* the screen was in. GUIs helped mitigate that problem by minimizing the number of state changes that the screen went through. In modern GUIs, a great deal of work is put in to keeping common features on the screen at all times and making sure the user does not get confused by hidden states.

It is ironic, then, that the code implementing these "stateless" GUIs, is strongly state driven. In such GUIs, the code must figure out which menu items and buttons to grey out, which subwindows should appear, which tab should be activated, where the focus ought to be put, etc. All these decisions relate to the state of the interface.

I learned a very long time ago that controlling these factors is a nightmare unless you organize them into a single control structure. That control structure is best characterized as an FSM. Since those days, I have been writing almost all my GUIs using FSMs generated by SMC (or its predecessors).

Consider the state machine in Listing 29-11. This machine controls the GUI for the login portion of an application. Upon getting a start event, the machine puts up a login screen. Once the user hits the enter key, the machine checks the password. If the password is good, it goes to the `loggedIn` state and starts the user process (not shown). If the password is bad, it displays a screen informing the user that his password is bad. If the user wants to try again, he hits the OK button; otherwise he hits the cancel button. If a bad password is entered three times in a row (`thirdBadPassword` event), the machine locks the screen until the administrator password is entered.

Listing 29-11

`login.sm`

```
Initial init
{
  init
  {
    start logginIn displayLoginScreen
  }

  logginIn
  {
    enter checkingPassword checkPassword
    cancel init clearScreen
  }

  checkingPassword
  {
    passwordGood loggedIn startUserProcess
    passwordBad notifyingPasswordBad displayBadPasswordScreen
    thirdBadPassword screenLocked displayLockScreen
  }

  notifyingPasswordBad
  {
    OK checkingPassword displayLoginScreen
    cancel init clearScreen
  }
```

```
  screenLocked
  {
    enter checkingAdminPassword checkAdminPassword
  }

  checkingAdminPassword
  {
    passwordGood init clearScreen
    passwordBad screenLocked displayLockScreen
  }
}
```

What we've done here is to capture the high-level policy of the application in a state machine. This high-level policy lives in one place and is easy to maintain. It vastly simplifies the rest of the code in the system because that code is not mixed with the policy code.

Clearly this approach can be used for interfaces other than GUIs. Indeed, I have used similar approaches for textual and machine–machine interfaces as well. But GUIs tend to be more complex than those others, so the need for them and the volume of them are greater.

GUI Interaction Controllers

Imagine you want to allow your users to draw rectangles on the screen. The gestures they use are as follows: First they click on the rectangle icon in the pallet window. Then they position the mouse in the canvas window at one corner of the rectangle. Then they press the mouse button and drag the mouse toward the desired second corner. While the user drags, an animated image of the potential rectangle appears on the screen. The user manipulates the rectangle to the desired shape by continuing to hold the mouse button down while dragging the mouse. When the rectangle is right, the user releases the mouse button. The program then stops the animation and draws a fixed rectangle on the screen.

Of course, the user can abort this at any time by clicking on a different pallet icon. If the user drags the mouse out of the canvas window, the animation disappears. If the mouse returns to the canvas window, the animation reappears.

Finally, having finished drawing a rectangle, the user can draw another simply by clicking and dragging again in the canvas window. There is no need to click on the rectangle icon in the pallet.

What I have described here is a finite state machine. The state transition diagram appears in Figure 29-6. The solid circle with the arrow denotes the starting state of the state machine.[7] The solid circle with the open circle around it is the final state of the machine.

GUI interactions are rife with finite state machines. They are driven by the incoming events from the user. Those events cause changes in the state of the interaction.

Distributed Processing

Distributed processing is yet another situation in which the state of the system changes based upon incoming events. For example, suppose you had to transfer a large block of information from one node on a network to another. Suppose also that network response time is precious, so you need to chop up the block and send it as a group of small packets.

The state machine depicting this scenario is shown in Figure 29-7. It starts by requesting a transmission session, proceeds by sending each packet and waiting for an acknowledgment, and finishes by terminating the session.

7. See "States and Internal Transitions" on page 493.

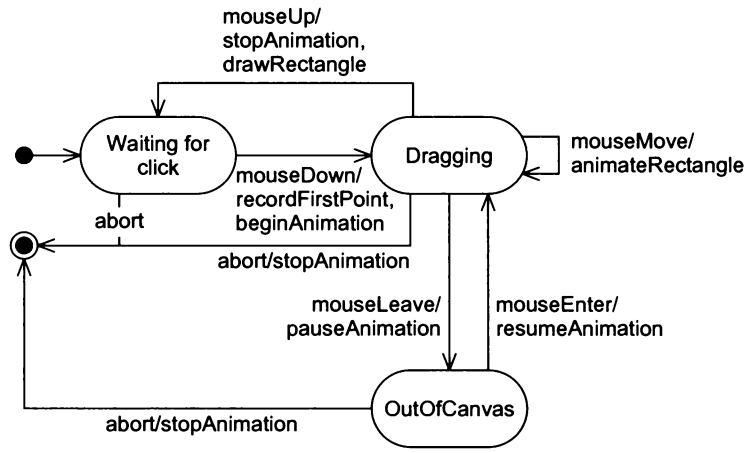


Figure 29-6 Rectangle Interaction State Machine

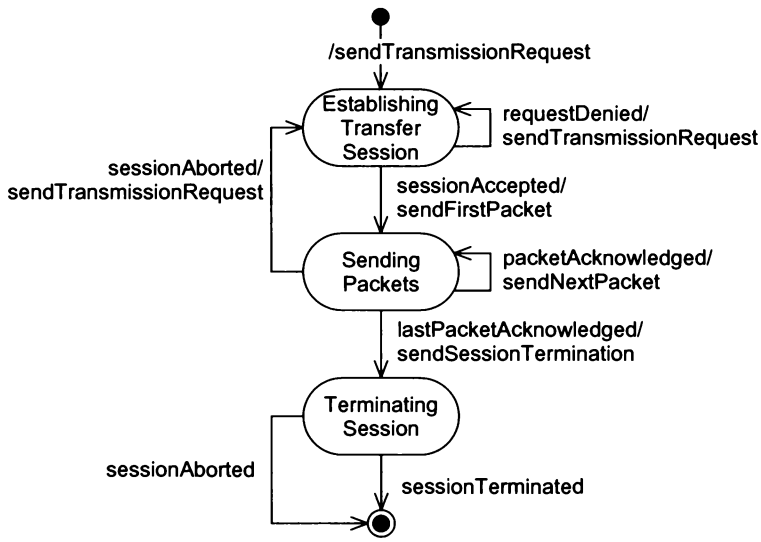


Figure 29-7 Sending large block using many packets

Conclusion

Finite state machines are underutilized. There are many scenarios in which their use would help to create clearer, simpler, more flexible, and more accurate code. Making use of the STATE pattern and simple tools for generating the code from state transition tables can be of great assistance.

Listings

`Turnstile.java` Using Table Interpretation

This listing shows how a finite state machine can be implemented by interpreting a vector of transition data structures. It is completely compatible with the `TurnstileController` in Listing 29-2 and the `TurnstileTest` in Listing 29-3.

Listing 29-12

Turnstile.java using table interpretation

```
import java.util.Vector;

public class Turnstile
{
  // States
  public static final int LOCKED = 0;
  public static final int UNLOCKED = 1;

  // Events
  public static final int COIN = 0;
  public static final int PASS = 1;

  /*private*/ int state = LOCKED;
  private TurnstileController turnstileController;
  private Vector transitions = new Vector();

  private interface Action
  {
    void execute();
  }

  private class Transition
  {
    public Transition(int currentState, int event,
                      int newState, Action action)
    {
      this.currentState = currentState;
      this.event = event;
      this.newState = newState;
      this.action = action;
    }

    int currentState;
    int event;
    int newState;
    Action action;
  }

  public Turnstile(TurnstileController action)
  {
    turnstileController = action;
    addTransition(LOCKED,   COIN, UNLOCKED, unlock()  );
    addTransition(LOCKED,   PASS, LOCKED,   alarm()   );
    addTransition(UNLOCKED, COIN, UNLOCKED, thankyou());
    addTransition(UNLOCKED, PASS, LOCKED,   lock()    );
  }

  private void addTransition(int currentState, int event,
                             int newState, Action action)
  {
    transitions.add(
      new Transition(currentState, event, newState, action));
  }
```

```
  private Action lock()
  {
    return new Action(){public void execute(){doLock();}};
  }

  private Action thankyou()
  {
    return new Action(){public void execute(){doThankyou();}};
  }

  private Action alarm()
  {
    return new Action(){public void execute(){doAlarm();}};
  }

  private Action unlock()
  {
    return new Action(){public void execute(){doUnlock();}};
  }

  private void doUnlock()
  {
    turnstileController.unlock();
  }

  private void doLock()
  {
    turnstileController.lock();
  }

  private void doAlarm()
  {
    turnstileController.alarm();
  }

  private void doThankyou()
  {
    turnstileController.thankyou();
  }

  public void event(int event)
  {
    for (int i = 0; i < transitions.size(); i++)
    {
      Transition transition = (Transition) transitions.elementAt(i);
      if (state == transition.currentState && event == transition.event)
      {
        state = transition.newState;
        transition.action.execute();
      }
    }
  }

}
```

`Turnstile.java` Generated by SMC, and Other Support Files

Listings 31-13 through 31-16 complete the code for the SMC example of the turnstile. Turnstile.java was generated by SMC. The generator creates a bit of cruft, but the code is not bad.

Listing 29-13

`Turnstile.java` (generated by SMC)

```
//---------------------------------------------
//
// FSM:       Turnstile
// Context:   TurnstileActions
// Exception: FSMError
// Version:
// Generated: Thursday 09/06/2001 at 12:23:59 CDT
//
//---------------------------------------------

//---------------------------------------------
//
// class Turnstile
//    This is the Finite State Machine class
//
public class Turnstile extends TurnstileActions
{
  private State itsState;
  private static String itsVersion = "";

  // instance variables for each state
  private static Locked itsLockedState;
  private static Unlocked itsUnlockedState;

  // constructor
  public Turnstile()
  {
    itsLockedState = new Locked();
    itsUnlockedState = new Unlocked();

    itsState = itsLockedState;

    // Entry functions for: Locked
  }

  // accessor functions

  public String getVersion()
  {
    return itsVersion;
  }

  public String getCurrentStateName()
  {
    return itsState.stateName();
  }

  // event functions - forward to the current State
```

```
public void pass() throws FSMError
{
  itsState.pass();
}

public void coin() throws FSMError
{
  itsState.coin();
}

//-----------------------------------------
//
// private class State
//    This is the base State class
//
private abstract class State
{
  public abstract String stateName();

  // default event functions

  public void pass() throws FSMError
  {
    throw new FSMError( "pass", itsState.stateName());
  }

  public void coin() throws FSMError
  {
    throw new FSMError( "coin", itsState.stateName());
  }

}

//-----------------------------------------
//
// class Locked
//    handles the Locked State and its events
//
private class Locked extends State
{
  public String stateName()
    { return "Locked"; }

  //
  // responds to coin event
  //
  public void coin()
  {
    unlock();

    // change the state
    itsState = itsUnlockedState;
  }
```

```
    //
    // responds to pass event
    //
    public void pass()
    {
      alarm();

      // change the state
      itsState = itsLockedState;
    }
  }

  //-----------------------------------------
  //
  // class Unlocked
  //    handles the Unlocked State and its events
  //
  private class Unlocked extends State
  {
    public String stateName()
      { return "Unlocked"; }

    //
    // responds to pass event
    //
    public void pass()
    {
      lock();

      // change the state
      itsState = itsLockedState;
    }

    //
    // responds to coin event
    //
    public void coin()
    {
      thankyou();

      // change the state
      itsState = itsUnlockedState;
    }
  }

}
```

`FSMError` is the exception that we told SMC to throw if there were ever an illegal event. The turnstile example is so simple that there can't be an illegal event, so the exception is useless. However, in larger state machines, there are events that should not occur in certain states. Those transitions are never mentioned in the input to SMC. Thus, if such an event were ever to occur, the generated code would throw the exception.

Listing 29-14

FSMError.java

```
public class FSMError extends Exception
{
  public FSMError(String event, String state)
```

```
  {
    super("Invalid event:" + event + " in state:" + state);
  }
}
```

The test code for the SMC generated state machine is very similar to all the other test programs we've written in this chapter. There are some minor differences.

Listing 29-15

SMCTurnstileTest.java

```
import junit.framework.*;
import junit.swingui.TestRunner;

public class SMCTurnstileTest extends TestCase
{
  public static void main(String[] args)
  {
    TestRunner.main(new String[]{"SMCTurnstileTest"});
  }

  public SMCTurnstileTest(String name)
  {
    super(name);
  }

  private TurnstileFSM t;
  private boolean lockCalled = false;
  private boolean unlockCalled = false;
  private boolean thankyouCalled = false;
  private boolean alarmCalled = false;

  public void setUp()
  {
    TurnstileController controllerSpoof =
      new TurnstileController()
    {
      public void lock() {lockCalled = true;}
      public void unlock() {unlockCalled = true;}
      public void thankyou() {thankyouCalled = true;}
      public void alarm() {alarmCalled = true;}
    };

    t = new TurnstileFSM(controllerSpoof);
  }

  public void testInitialConditions()
  {
    assertEquals("Locked", t.getCurrentStateName());
  }

  public void testCoinInLockedState() throws Exception
  {
    t.coin();
    assertEquals("Unlocked", t.getCurrentStateName());
    assert(unlockCalled);
  }
```

```
  public void testCoinInUnlockedState() throws Exception
  {
    t.coin(); // put in Unlocked state
    t.coin();
    assertEquals("Unlocked", t.getCurrentStateName());
    assert(thankyouCalled);
  }

  public void testPassInLockedState() throws Exception
  {
    t.pass();
    assertEquals("Locked", t.getCurrentStateName());
    assert(alarmCalled);
  }

  public void testPassInUnlockedState() throws Exception
  {
    t.coin(); // unlock
    t.pass();
    assertEquals("Locked", t.getCurrentStateName());
    assert(lockCalled);
  }
}
```

The `TurnstileController` class is identical to all the others that appeared in this chapter. You can see it in Listing 29-2.

The `ant` file used to generate the `Turnstile.java` code is shown in Listing 29-16. Note that it's not a very big deal. Indeed, if you wanted to simply type the build command in a DOS window, you could type:

```
java smc.Smc -f TurnstileFSM.sm
```

Listing 29-16

build.xml

```
<project name="SMCTurnstile" default="TestSMCTurnstile" basedir=".">

  <property environment="env" />

  <path id="classpath">
    <pathelement path="${env.CLASSPATH}"/>
  </path>

  <target name="TurnstileFSM">
    <java classname="smc.Smc">
      <arg value="-f TurnstileFSM.sm"/>
      <classpath refid="classpath" />
    </java>
  </target>

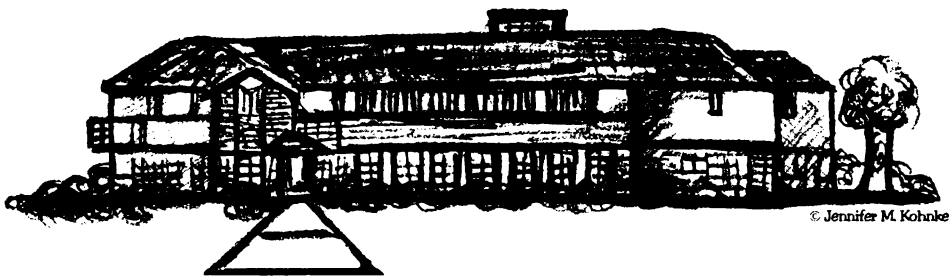
</project>
```

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Coplien and Schmidt. *Pattern Languages of Program Design*. Reading, MA: Addison–Wesley, 1995.

30

The ETS Framework



By Robert C. Martin and James Newkirk

This chapter describes a significant software project that was developed from March 1993 until late 1997. The software was commissioned by the Educational Testing Service (ETS) and was developed by the two of us and several other developers at Object Mentor, Inc.

Our focus in this chapter is on the techniques, both technical and managerial, for producing a reusable framework. The creation of such a framework was an essential step in the success of the project, and the design and history of its development should prove educational.

No software project is developed in a perfect environment, and this one was no exception. In order to understand the technical aspects of the design, it is important to consider the environmental issues as well. Therefore, before we dive into the software engineering aspects of the project, we are going to give you a bit of background on the project and the environment in which it was developed.

Introduction

Project Overview

To become a licensed architect in the United States or Canada, you must pass an examination. If you pass, a state licensing board will give you a license to practice architecture. The examination was developed by the Educational Testing Service (ETS) under a charter by the National Council of Architectural Registration Boards (NCARB), and it is currently administered by the Chauncey Group International.

The test consists of nine divisions and is taken over a period of several days. In the three graphical divisions of the examination, candidates are asked to create solutions in a CAD-like environment by drawing or placing objects. For example, they might be asked to do the following:

- Design the floor plan of a certain kind of building
- Design a roof to fit on top of an existing building
- Place a proposed building on a parcel of land and design the parking area, road system, and walkway system to serve the building

In the past, the candidates' solutions to these problems were drawn using pencil and paper. These documents were then given to a cadre of jurors for scoring. The jurors were highly experienced architects who would pore over the candidates' solutions and decide whether to pass or fail them.

In 1989, NCARB commissioned ETS to research whether or not an automated system for delivering and scoring the graphical portions of the examination could be developed. By 1992, the ETS and the NCARB had agreed that such a system was indeed possible. Furthermore, they felt that an object-oriented approach would be appropriate due to the constantly changing requirements. So they contacted Object Mentor, Inc. (OMI) to help with the design.

In March 1993, OMI was granted a contract to produce a portion of the test. One year later, after successfully completing that portion, OMI was granted a second contract to produce the bulk of the remainder.

Program Structure. The structure that ETS decided on was quite elegant. The graphics examination would be broken into 15 different problems, called vignettes. Each vignette tested a particular area of knowledge. One might test the candidate's understanding of roof design, and another might test their understanding of how to design floor plans.

Each vignette was further subdivided into two sections. The "delivery" section was to be a graphical user interface upon which the candidate would "draw" the solution to the problem at hand. The "scoring" section was to read the solution created by the delivery section and score it. Delivery would be conducted at a location convenient to the candidate. The solutions would then be transmitted to a central location where they would be scored.

Scripts. Although there were only 15 vignettes, each would have many possible "scripts." A script would specify the exact nature of the problem that the candidate would solve. For example, the floor-plan vignette might have one script that asked the candidate to design a library, and another that asked the candidate to design a grocery store. Thus, the vignette programs had to be written in a generic fashion. They had to be delivered and scored in a way that was governed by the script.

Platform. Both the delivery and scoring programs were to run under Windows 3.1 (later upgraded to W95/NT). The programs were to be written in C++ using object-oriented techniques.

First Contract. In March 1993, OMI was granted a contract to develop the delivery and scoring of the most complex of all the vignettes: "Building Design." This decision was motivated by Booch's recommendation to develop the highest-risk elements first as a way to manage risk and calibrate the estimation process of the team.

Building Design. Building Design was to test the candidate's ability to design the floor plan of a relatively simple two-story building. The candidate would be given a building to design, complete with requirements and constraints. The candidate would then use the delivery program to place rooms, doors, windows, corridors, stairs, and elevators into their solution.

The scoring program would then check the solution against a large number of "features" that would assess the candidate's knowledge. The nature of these features is confidential, but in general terms, they would assess things such as the following:

- Does the building meet the requirements of the client?
- Is the building code compliant?
- Has the candidate demonstrated design logic?
- Are the building and its rooms oriented correctly on the site?

Early History 1993–1994

At this early stage, two of us (Martin and Newkirk) were the sole developers working on this project. Our contract with ETS said that we were to produce both the delivery and scoring programs for Building Design. However, we also wanted to develop a reusable framework in conjunction with Building Design.

By 1997, there had to be 15 vignette delivery and scoring programs operational. That gave us four years. We felt that a reusable framework would give us a big advantage toward that goal. Such a framework would also go a long way toward helping us manage the consistency and quality of the vignettes. After all, we didn't want similar features in different vignettes to operate in subtly different ways.

So, in March 1993, we set out to produce the two components of Building Design and also a framework that could be reused by the remaining fourteen vignettes.

Success. In September 1993, the first versions of the delivery and scoring programs were complete, and we demonstrated these programs to representatives of NCARB and ETS. These demonstrations were well received, and a field trial was scheduled for January 1994.

As with most projects, once users see the program actually operating, they realize that what they asked for was not really what they wanted. We had been sending interim versions of the vignette to ETS every week throughout 1993, and there had been a large number of changes and enhancements made right up to that September demonstration.

After the demonstration, the looming field trial caused the rate of changes and enhancements to reach deluge proportions. The two of us were kept busy, full time, making and testing these changes and preparing the programs for the field trial.

This churning of the specification for Building Design was accelerated still further by the results of the field trial, keeping us even busier through the first quarter of 1994.

In December 1993, negotiations were begun for a contract to build the rest of the vignettes. These negotiations would take three months to close. In March 1994, ETS granted OMI a contract to produce a framework and 10 more vignettes. ETS's own engineers would produce the remaining five vignettes based upon our framework.

Framework?

Late in 1993, at the height of the churn in Building Design, one of us (Newkirk) spent a week with one of the engineers at ETS to prepare him for his part in the upcoming contract. The objective was to demonstrate how the code within the 60,000-line, reusable C++ framework could be reused to build other vignettes. However, things did not go well. By the end of the week, it was clear that the only way to reuse the framework was to cut and paste bits and pieces of its source code into the new vignettes. Clearly this was not a good option.

In hindsight, there were two reasons for our failure to create a workable framework. First, we had been focusing on Building Design to the exclusion of all the other vignettes. Second, we went through months of churning requirements and schedule pressure. These two things together allowed concepts that were specific to Building Design to infiltrate the framework.

To a certain extent, we naively took the benefits of object-oriented technology for granted. We felt that by using C++ and by doing a careful object-oriented design, a reusable framework would be easy to create. We were wrong. We discovered what had been known for years—building reusable frameworks is *hard.*

Framework!

In March 1994, after the new contract was signed, we added two more engineers to the project and began developing the new vignettes. We still believed we needed a framework and were convinced that what we had would not serve that function. It was clear we needed to change our strategy.

The 1994 Team

- Robert C. Martin, architect and lead designer, 20+ years experience
- James W. Newkirk, designer and project leader, 15+ years experience
- Bhama Rao, designer and programmer, 12+ years experience
- William Mitchell, designer and programmer, 15+ years experience

The Deadline

The deadline was set by the fact that the test was to go into production in 1997. Candidates would be tested in February and scored in May. This was an absolute requirement.

The Strategy

To make that schedule and to ensure that the quality and consistency of the program could be managed, we adopted a new strategy for the construction of the framework. Portions of the original 60,000-line framework were preserved, *but the majority was scrapped.*

A Rejected Alternative. One option would have been to try to redesign the framework up front and complete it before any of the vignettes were begun. Indeed, many people would identify this with an architecture-driven approach. However, we chose not to pursue this option, because it would have caused us to produce large amounts of framework code that could not be tested inside working vignettes. We did not trust our ability to completely anticipate the needs of the vignettes. In short, we felt that the architecture needed almost immediate verification by being used in working vignettes. We did not want to guess.

Rebecca Wirfs–Brock once said, "You have to build at least three or more applications against a framework (and then throw them away) before you can be reasonably confident that you have built the right architecture for that domain."[2] After failing to produce a framework, we felt similarly. Thus, we decided that we would develop the framework concurrently with the development of several new vignettes. This would allow us to compare similar features of the vignettes and design those features in a generic and reusable way.

Four vignettes were begun in parallel. As they were being developed, certain portions were found to be similar. These were then refactored into more generic form and refit into all three vignettes. Thus, nothing entered the framework unless it had been successfully reused in at least four vignettes.

Also, portions of Building Design were excised and refactored in a similar fashion. Once these portions were working in all three vignettes, they were placed in the framework.

Among the common features that were added to the framework were the following:

- The structure of the UI screen–Message windows, drawing windows, button palettes, etc.
- Creating, moving, adjusting, identifying, and deleting graphic elements

2. [BOOCH-OS], p. 275.

- Zooming and scrolling
- Drawing simple sketch elements such as lines, circles, and polylines
- Vignette timing and automatic abort
- Saving and restoring solution files, including error recovery
- Mathematical models of many geometric elements: line, ray, segment, point, box, circle, arc, triangle, polygon, etc. These models include methods such as Intersection, Area, `IsPointIn`, `IsPointOn`, etc.
- Evaluation and weighting of individual scoring features.

Over the next eight months, the framework grew to some 60,000 lines of C++ code, representing somewhat more than one man–year of direct effort. But this framework was being reused by four different vignettes.

Results

Throwing One Away. What were we to do with the old Building Design? As the framework grew and the new vignettes were successfully reusing it, Building Design stood more and more as an outsider. It was unlike all the other vignettes and would have to be maintained and evolved through separate means. Even though Building Design represented over a man–year of effort, we decided to be ruthless and discard the old version completely. We committed ourselves to redesign and reimplement it later in the project cycle.

Long Initial Development Time. A negative result of our framework strategy was the relatively long development time for the first vignettes. The first four vignette delivery programs required nearly four man–years to develop.

Reuse Efficiency. Upon completion of the initial vignettes, the framework was richly endowed with 60,000 lines of C++ code, and the vignette delivery programs were remarkably small. Each program had approximately 4,000 lines of boiler-plate code (i.e., code that is the same for each vignette). Each program also had an average of 6,000 lines of application-specific code. The smallest vignette had as few as 500 lines of application-specific code, and the largest had as many as 12,000. We felt that it was remarkable that, on average, nearly five-sixths of the code in each vignette was pulled from the framework. Only one-tenth of the code within these programs was unique.

Development Productivity. After the first four vignettes, development time drastically decreased. Seven more delivery programs (including a rewrite of Building Design) were completed within 18 man–months. The line-count ratios of these new vignettes remained roughly the same as the first four.

Moreover, the Building Design program, which took us over one man–year to write the first time, took only 2.5 man–months to rewrite from scratch with the framework in place. This is nearly a 6:1 increase in productivity.

Another way to look at these results is that the first five vignettes, including Building Design, required one man–year of effort apiece. Subsequent vignettes, however, required 2.6 man–months apiece—an increase of nearly 400 percent.

Weekly Delivery. From the outset of the project, and throughtout its active development, we shipped interim versions to ETS every week. ETS would test and evaluate these versions and then send us a list of changes. We would estimate these changes, and then work with ETS to schedule the week in which they'd be delivered. Difficult changes, or changes of low importance, were often delayed in preference to high priority changes. Thus, ETS kept control of the project and schedule throughout.

A Robust and Flexible Design. One of the most satisfying aspects of the project is the way that the architecture and framework have weathered the intense flux of requirements changes. During the peak of development, nary a week went by without a long list of changes and fixes being identified. Some of these changes were in

response to bugs, but many more were due to changes in the actual requirements. Yet, with all the modifications, fiddling, and tweaking in the midst of heavy development, "the design of the software didn't unravel."[3]

Final Outcome. By February 1997, architectural candidates began using the delivery programs to take their registration exams. By May 1997, their results began to be scored. The system has been up and running since that time, and it has worked well. Every architect candidate in North America now takes this exam using this software.

Framework Design

The Common Requirements of the Scoring Applications

Consider the problem of how to test someone's knowledge and skill. The scheme that was adopted by ETS for the NCARB programs is quite elaborate. We can illustrate it here by exploring a simple, fictitious example — a test for basic math.

In our basic math test, students are presented with 100 math problems ranging from simple addition and subtraction problems to large multiplication and long-division problems. We will examine their responses to these problems to ascertain their competency and skill in basic math. Our goal is to give them a pass/fail grade. "Pass" means that we are certain that they have acquired the fundamental knowledge and skill needed for basic math. "Fail" means that we are sure that they have not acquired this knowledge and skill. In those cases where we are unsure, we will return a score of "indeterminate."

However, we have another goal, too. We want to be able to enumerate the strengths and weaknesses of the student. We want to partition the topic of basic math into subtopics and then evaluate the student for each of those subtopics.

Consider, for example, the problem of a person who has learned an improper multiplication fact. Perhaps they always mistake 7×8 for 42. This person will get a large percentage of the multiplication and division problems wrong. Certainly that student deserves to fail the test. On the other hand, consider that the student did *everything* else correctly! The student correctly created the partial products in long multiplication and correctly structured the long-division problems. Indeed, the *only* mistake that the student made was $7 \times 8 = 42$. Certainly we'd like to know that. Indeed, since the corrective action for such a student is so simple, we might very well want to give the student a passing grade along with some corrective guidance.

So how can we structure the scoring of the test to determine the areas of basic math in which the student has gained expertise and those areas in which he or she has not? Consider the diagram in Figure 30-1. The rectangles on this diagram show areas of expertise that we want to test for. The lines show a hierarchical dependency. Thus, knowledge of basic math depends on knowledge of terms and factors. Knowledge of terms depends on knowledge of addition facts plus the mechanics of addition and subtraction. Knowledge of addition depends upon knowledge of the commutative and associative properties of addition and the mechanics of carrying.

The leaf rectangles are called "features." Features are units of knowledge that can be evaluated and given acceptable (A), unacceptable (U), or indeterminate (I) values. Thus, given our 100 problems and the student's answers, we want to apply each feature to every problem and determine a score. In the case of the "Carry" feature, we would look at every addition problem and compare it to the student's answer. If the student got all the addition problems correct, the result of the "Carry" feature would, of course, be 'A.' However, for each addition problem that the student got wrong, we would try to determine if the error made was an error in carrying. We might try different combinations of carrying errors to see if one of those errors would lead to the answer that the student gave. If we could determine, with a high degree of probability, that a carry error was made, we would adjust the score of the carry feature accordingly. In the end, the score returned by the carry feature would be a statistical result based upon the total number of wrong answers that could be traced to errors in carrying.

3. Pete Brittingham, NCARB Project Manager, ETS.

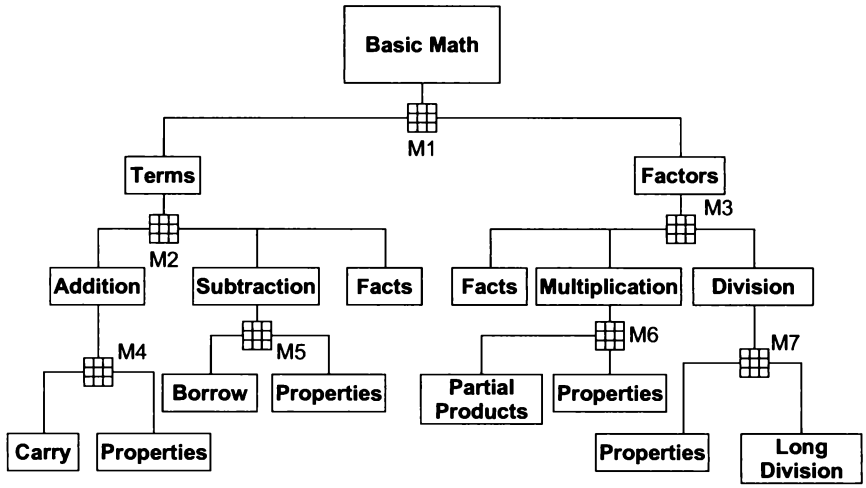


Figure 30-1 Feature Hierarchy for Basic Math

If, for example, the student got half the addition problems wrong, and most of them could be traced to carry errors, we would certainly return a 'U' for the carry feature. If, on the other hand, only a quarter of the errors could be traced to carry errors, we might return an 'I.'

In the end, all the features are evaluated in this manner. Each feature examining the test answers to develop a score for that particular feature. The scores of the various features represent an analysis of the student's knowledge of basic math.

The next step is to derive a final grade from that analysis. To do this, we merge the scores of the features up the hierarchy through the use of weights and matrices. Notice on Figure 30-1 that there are matrix icons at the junctures between the levels of the hierarchy. The matrix associates a weighting factor with the score from each feature and then provides a map for the score for that level in the hierarchy. For example, the matrix just below the Addition node would establish the weights to be applied to the Carry and Properties scores, and would then describe the mapping that would generate the overall score for Addition.

Figure 30-2 shows the form of one of these matrices. The input from Carry is considered more important that the input from Properties, so it is given twice the weight. The weighted scores are then added together, and the result is applied to the matrix.

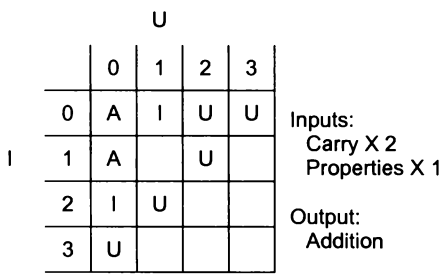


Figure 30-2 The Addition Matrix

For example, let's presume a score of 'I' from Carry, and a score of 'A' from Properties. There are no 'U' scores, so we use the left-most column of the matrix. The weighted 'I' score is 2, so we use the third row of the matrix, giving a result of 'I.' Notice that there are holes in the matrix. These are impossible conditions. Given the current weightings, there is no combination of scores that will select the empty cells in the matrix.

This scheme of weights and matrices is repeated at each level of the hierarchy until a final score is derived. Thus, the final score is a merging and remerging of the various feature scores. The structure of the hierarchy lends itself to very precise tuning on the part of the psychometricians at ETS.

The Design of the Scoring Framework

Figure 30-3 shows the static structure of the scoring framework. The structure can be divided into two major sections. The three classes on the right, shown in distinctive font, are not part of the framework. They represent the classes that must be written for each specific scoring application. The rest of the classes in Figure 30-3 are framework classes, which are common to all scoring applications.

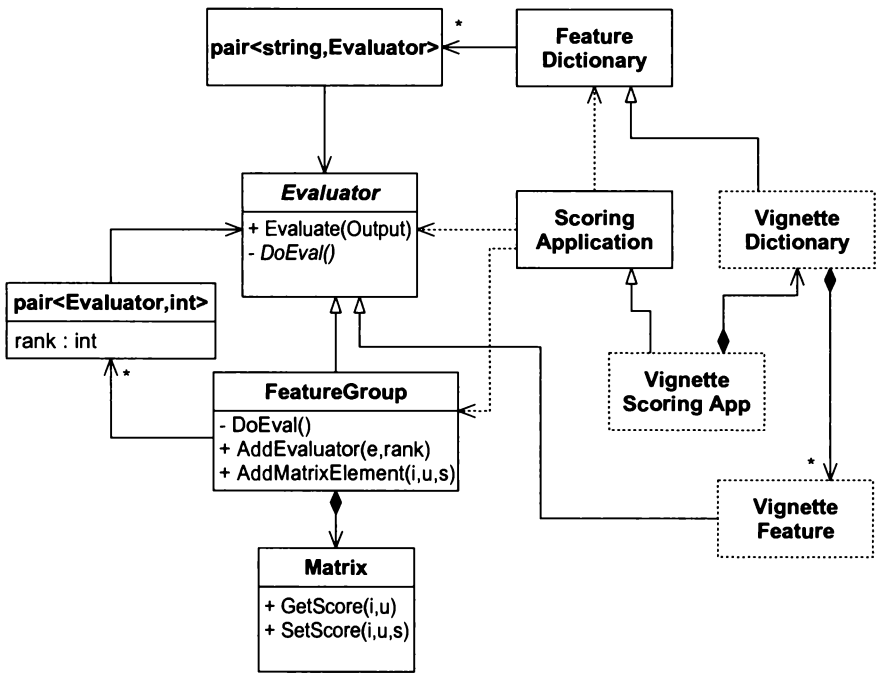


Figure 30-3 Scoring Framework

The most important class in the scoring framework is `Evaluator`. This class is an abstract class that represents both the leaf nodes and the matrix nodes of the scoring tree. The `Evaluate(ostream&)` function is called when the score of a node in the scoring tree is desired. This function makes use of the TEMPLATE METHOD[4] pattern in order to provide a standard method for logging scores to an output device.

Listing 30-1

`Evaluator`

```
class Evaluator
{
  public:
    enum Score {A,I,U,F,X};
    Evaluator();
    virtual ~Evaluator();

    Score Evaluate(ostream& scoreOutput);
```

4. [GOF95], p. 325.

```
    void SetName(const String& theName) {itsName = theName;}
    const String& GetName() {return itsName;}

  private:
    virtual Score DoEval() = 0;

    String itsName;
};
```

See Listings 30-1 and 30-2. The `Evaluate()` function calls a private, pure virtual function called `DoEval()`. This function will be overridden to perform the actual evaluation of the scoring-tree node. It returns the score and allows `Evaluate()` to output it in the standard form.

Listing 30-2

`Evaluator::Evaluate`

```
Evaluator::Score Evaluator::Evaluate(ostream& o)
{
    static char scoreName[] = {'A', 'I', 'U', 'F', 'X'};
    o << itsName << ":";
    score = DoEval();
    o << scoreName[score] << endl;
    return score;
}
```

Leaf nodes of the scoring tree are represented by the `VignetteFeature` class in Figure 30-3. Actually, there would be dozens of such classes in each scoring application. Each would override `DoEval()` to calculate the score for its own particular scoring feature.

Matrix nodes of the scoring tree are represented by the `FeatureGroup` class in Figure 30-3. Listing 30-3 shows what this class looks like. There are two functions that assist in creating a `FeatureGroup` object. The first is `AddEvaluator`, and the second is `AddMatrixElement`.

Listing 30-3

`FeatureGroup`

```
class FeatureGroup : public Evaluator
{
public:
  FeatureGroup(const RWCString& name);
  virtual ~FeatureGroup();

  void AddEvaluator(Evaluator* e , int rank);

  void AddMatrixElement(int i, int u, Score s);
private:
  Evaluator::Score DoEval();
  Matrix itsMatrix;
  vector<pair<Evaluator*,int> > itsEvaluators;
};
```

The `AddEvaluator` function allows child nodes to be added to the `FeatureGroup`. For example, referring back to Figure 30-1, the `Addition` node would be a `FeatureGroup`, and we would call `AddEvaluator` twice to

load the `Carry` and `Properties` nodes to it. The `AddEvaluator` function allows the rank of the evaluator to be specified. The rank is the multiplier that gets applied to the score coming out of the evaluator. Thus, when we called `AddEvaluator` to add `Carry` to the `Addition FeatureGroup`, we would have specified a rank of 2 because the `Carry` feature has twice the weight of the `Properties` feature.

The `AddMatrixElement` function adds a cell to the matrix. It must be called for every cell that requires population. For example, the matrix in Figure 30-2 would be created using the sequence of calls in Listing 30-4.

Listing 30-4

Creation of the Addition Matrix

```
addition.AddMatrixElement(0,0,Evaluator::A);
addition.AddMatrixElement(0,1,Evaluator::I);
addition.AddMatrixElement(0,2,Evaluator::U);
addition.AddMatrixElement(0,3,Evaluator::U);
addition.AddMatrixElement(1,0,Evaluator::A);
addition.AddMatrixElement(1,2,Evaluator::U);
addition.AddMatrixElement(2,0,Evaluator::I);
addition.AddMatrixElement(2,1,Evaluator::U);
addition.AddMatrixElement(3,0,Evaluator::U);
```

The `DoEval` function simply iterates through the list of evaluators, multiplying their score by the rank and adding the product to the appropriate accumulators for I and U scores. Once complete, it uses those accumulators as matrix indices to pull out the final score. (See Listing 30-5.)

Listing 30-5

FeatureGroup::DoEval

```
Evaluator::Score FeatureGroup::DoEval()
{
    int sumU, sumI;
    sumU = sumI = 0;
    Evaluator::Score s, rtnScore;
    Vector<Pair<Evaluator*, int> >::iterator ei;
    ei = itsEvaluators.begin()

    for(; ei != itsEvaluators.end(); ei++)
    {
      Evaluator* e = (*ei).first;
      int rank = (*ei).second;

      s = e.Evaluate(outputStream);

      switch(s)
      {
      case I:
        sumI += rank;
      break;
      case U:
        sumU += rank;
      break;
      }
    } // for ei
    rtnScore = itsMatrix.GetScore( sumI, sumU );
    return rtnScore;
}
```

One last issue remains. How does the scoring tree get built? It was quite evident that the psychometricians at ETS would want the ability to change the topology and weighting of the tree without having to change the actual applications. Thus the scoring tree is built up by the `VignetteScoringApp` class. (See Figure 30-3.)

Each scoring application had its own implementation for this class. One of the responsibilities of this class was to build a derivative of the `FeatureDictionary` class. This class contained a mapping of strings to `Evaluator` pointers.

When a scoring application was started, the scoring framework gained control. It invoked the method in the `ScoringApplication` class that caused the appropriate derivative of the `FeatureDictionary` to be created. It then read a special text file that described the topology of the scoring tree and its weights. This text file identified features by using special names. These names were the names that were associated with the appropriate `Evaluator` pointers in the `FeatureDictionary`.

Thus, in its simplest form, a scoring application was nothing more than a set of features and a method that constructed a `FeatureDictionary`. The building and evaluation of the scoring tree was handled by the framework and was therefore common to all the scoring applications.

A Case for TEMPLATE METHOD

One of the vignettes tested the candidates' abilities to lay out the floor plan of a building such as a library or a police station. In this vignette, the candidate had to draw rooms, corridors, doors, windows, wall openings, stairs, elevators, etc. The program converted the drawing into a data structure that the scoring program could interpret. The object model looked something like Figure 30-4.

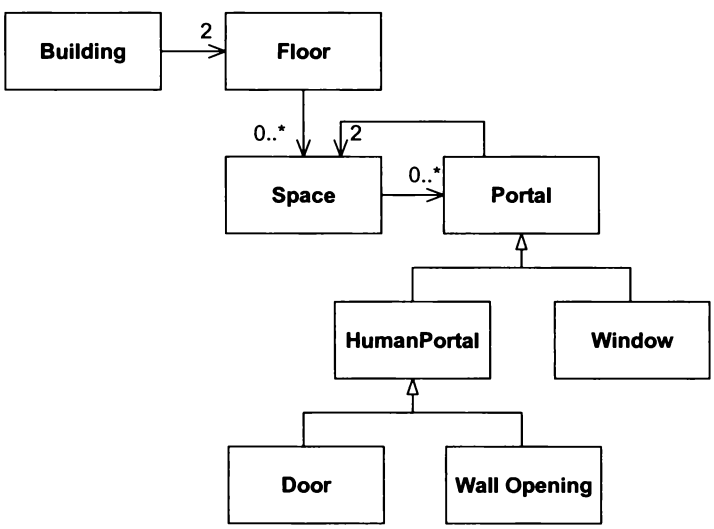


Figure 30-4 Floor Plan Data Structure

Make no mistake about this. The objects in this data structure had very minimal functionality. They were not polymorphic objects in any sense of the word. Rather, they were simple data carriers—A purely representational model.

A building was composed of two floors. Each floor had many spaces. Each space contained many portals, each of which separated two spaces. Portals could be windows or could allow a human to pass through. Human portals were either wall openings or doors.

Scoring was done by testing the solution for a set of *features*. The features were things like the following:

- Did the candidate draw all the required spaces?
- Does each space have an acceptable aspect ratio?

- Does each space have a way in?
- Do external spaces have windows?
- Are the men's and ladies' washrooms connected by a door?
- Does the president's office have a view of the mountains?
- Does the kitchen have easy access to a back alley?
- Does the lunchroom have easy access to the kitchen?
- Can a person navigate the corridor system and get to each room?

The psychometricians at ETS wanted to be able to easily change the shape of the scoring matrix.They wanted to be able to change the weightings, regroup the features into different subhierarchies, etc. They wanted to be able to take out features that they considered worthless or add new features. Most of these manipulations were a matter of changing a single configuration text file.

For performance reasons, we only wanted to calculate the features that were included in the matrix. So we created classes for each feature. Each of these `Feature` classes had an `Evaluate` method that would walk the data structure in Figure 30-4 and calculate a score. This meant that we had dozens and dozens of `Feature` classes that all walked the same data structure. The code duplication was horrendous.

Write a Loop Once

To deal with the code duplication, we started using the TEMPLATE METHOD pattern. This was in 1993 and 1994, long before we knew anything about patterns. We called what we were doing "Write a loop once." (See Listings 30-6 and 30-7.) These are the actual C++ modules from that program.

Listing 30-6

solspcft.h

```
/* $Header: /Space/src_repository/ets/grande/vgfeat/
solspcft.h,v 1.2 1994/04/11 17:02:02 rmartin Exp $ */

#ifndef FEATURES_SOLUTION_SPACE_FEATURE_H
#define FEATURES_SOLUTION_SPACE_FEATURE_H

#include "scoring/eval.h"

template <class T> class Query;

class SolutionSpace;
//-----------------------------------------
// Name
//  SolutionSpaceFeature
//
// Description
//  This class is a base class which provides a loop which
//  scans through the set of solution spaces and then
//  finds all the solution spaces that match it.  Pure virtual
//  functions are provided for when a solution space are found.
//

class SolutionSpaceFeature : public Evaluator
{
  public:
    SolutionSpaceFeature(Query<SolutionSpace*>&);
```

```
    virtual ~SolutionSpaceFeature();
    virtual Evaluator::Score DoEval();
    virtual void NewSolutionSpace(const SolutionSpace&) = 0;
    virtual Evaluator::Score GetScore() = 0;

  private:
    SolutionSpaceFeature(const SolutionSpaceFeature&);
    SolutionSpaceFeature& operator= (const SolutionSpaceFeature&);

    Query<SolutionSpace*>& itsSolutionSpaceQuery;
};
#endif
```

Listing 30-7

solspcft.cpp

```
/* $Header: /Space/src_repository/ets/grande/vgfeat/
solspcft.cpp,v 1.2 1994/04/1 1 17:02:00 rmartin Exp $ */

#include "componen/set.h"

#include "vgsolut/solspc.h"
#include "componen/query.h"
#include "vgsolut/scfilter.h"
#include "vgfeat/solspcft.h"

extern ScoringFilter* GscoreFilter;

SolutionSpaceFeature::SolutionSpaceFeature(Query<SolutionSpace*>& q)
: itsSolutionSpaceQuery(q) {}

SolutionSpaceFeature::~SolutionSpaceFeature() {}

Evaluator::Score SolutionSpaceFeature::DoEval()
{
  Set<SolutionSpace*>& theSet = GscoreFilter->GetSolutionSpaces();
  SelectiveIterator<SolutionSpace*>ai(theSet,itsSolutionSpaceQuery);

  for (; ai; ai++)
  {
    SolutionSpace& as = **ai;
    NewSolutionSpace(as);
  }
  return GetScore();
}
```

As you can see from the comment header, this code was written in 1994. So it'll look a bit strange to those of you who are used to STL. Still, if you ignore the cruft and the bizarre iterators, you'll see the classic TEMPLATE METHOD pattern in there. The `DoEval` function loops through all the `SolutionSpace` objects. It then calls the pure virtual `NewSolutionSpace` function. Derivatives of `SolutionSpaceFeature` implement `NewSolutionSpace` and measure each space against a particular scoring criterion.